Smallest Substring Containing

AlgoExpert · Very Hard · Strings

The problem

- Shortest substring of `big` containing every character of `small` (with multiplicity), or "".
- Sliding window.

Intuition

- A sliding window expands until it covers all needed characters, then shrinks to the minimum.

Brute force vs optimal

- Check every substring → $O(n^2 \cdot m)$
- Sliding window with need counts → $O(n + m)$

Approach

- 1. Count needed characters
- 2. Expand right updating coverage
- 3. Shrink left while still covering
- 4. Track the smallest window

Worked example

```
big: abcd$ef$axb$c$

small: $$abf
```

Smallest window containing all chars of small

Worked example

```
expand r until nothing is missing

then shrink l while still valid
```

Expand right, shrink left

Worked example

```
answer: f$axb$
```

Answer: faxb

Java solution

```java
public class SmallestSubstringContaining {
    public static String smallestSubstringContaining(String big, String small) {
        Map<Character, Integer> need = new HashMap<>();
        for (char c : small.toCharArray()) need.merge(c, 1, Integer::sum);
        int missing = small.length(), l = 0, bs = 0, be = Integer.MAX_VALUE;
        for (int r = 0; r < big.length(); r++) {
            char c = big.charAt(r);
            if (need.containsKey(c)) { if (need.get(c) > 0) missing--; need.merge(c, -1, Integer::sum); }
            while (missing == 0) {
                if (r - l < be - bs) { bs = l; be = r; }
                char d = big.charAt(l++);
                if (need.containsKey(d)) { need.merge(d, 1, Integer::sum); if (need.get(d) > 0) missing++; }
            }
        }
        return be == Integer.MAX_VALUE ? "" : big.substring(bs, be + 1);
    }
}
```

Complexity

- Time: O(b + s)
- Space: O(s)

Edge cases & pitfalls

- Multiplicity matters
- None → ''
- Track 'missing' to avoid map scans

Interview tips & follow-ups

- Same as Minimum Window Substring

Recap

- A sliding window expands until it covers all needed characters, then shrinks to the minimum.
- Complexity: O(b + s)